SIMATS Engineering

Department of Information Technology

MLA0402 – DEEP LEARNING

ASSIGNMENT 2 – CO2

Deep Learning Architectures, Representation Learning and Unsupervised Neural Networks

Scenario: Smart Hospital Chest X-ray Screening

Name: SK Fazul Register Number: 192411063

Total Marks: 50

TASK 1 – Machine Learning and Deep Learning (3 Marks)

a) Differentiate machine learning and deep learning in terms of feature extraction, data requirement, computational cost and interpretability.

Classical machine learning (ML) and deep learning (DL) both learn a mapping from inputs to outputs, but they differ in who designs the features and how much data and computation they need. In the hospital's current pipeline, engineers design intensity histograms and edge counts and an SVM classifies them. In a deep network, the features are learned directly from the pixels.

Aspect	Machine Learning (histogram/edge features + SVM)	Deep Learning (network on raw pixels)
Feature extraction	Manual. Engineers design features before training, so quality depends on domain knowledge.	Automatic. Many layers learn a hierarchy of features (edges → textures → parts → lesions) from the data.
Data requirement	Works reasonably with hundreds to a few thousand labelled samples because the feature space is small.	Needs large amounts of data, because millions of parameters must be estimated; unlabelled data can also be used.
Computational cost	Low. Training runs on a CPU in minutes.	High. Training needs GPUs and many iterations; inference is also heavier.
Interpretability	Higher. Each feature (for example, an edge count)	Lower. It behaves like a black box, and learned

Aspect	Machine Learning (histogram/edge features + SVM)	Deep Learning (network on raw pixels)
	has a clear meaning and SVM decisions can be traced to support vectors.	features are distributed; tools such as saliency maps or Grad-CAM are needed to explain predictions.

b) Explain why the classical pipeline may fail to capture subtle lesion patterns in raw X-ray images, and how a deep network addresses this limitation.

A classical pipeline is limited by the features chosen in advance. Intensity histograms are global summaries that discard where the intensities occur, so a small, faint nodule changes the histogram very little. Edge counts respond to any boundary (ribs, vessels, noise, the heart outline) and cannot separate a lesion edge from normal anatomy. Subtle lesions are small, have low contrast, vary in shape and may appear anywhere in the lung. Any information that the hand-designed features do not encode is lost before the SVM sees it, and the SVM cannot recover it.

A deep network removes this bottleneck by learning the features and the classifier together, end to end. Early layers learn local detectors such as edges and gradients, middle layers combine them into textures and shapes, and deeper layers combine these into lesion-like patterns. Because the features are trained to minimise the classification loss, they are tuned to the lesion patterns that actually separate normal from abnormal scans, including patterns that engineers would not think to design.

c) With only 1,000 labelled scans, state one risk of training a deep network from scratch and one way to reduce that risk.

Risk: overfitting. A deep network has far more parameters than 1,000 labelled examples, so it can memorise the training scans (including their noise) and give high training accuracy but poor accuracy on new patients.

Mitigation: use the 100,000 unlabelled scans for unsupervised

pre-training (for example, an auto encoder) and then fine-tune on the 1,000 labelled scans. Data augmentation, dropout, weight decay and early stopping further reduce overfitting.

TASK 2 – Representation Learning (4 Marks)

a) Define representation learning and explain why raw pixel values are a poor representation for classification.

Representation learning is the set of methods by which a model automatically discovers, from data, the representation (features) needed for a task, instead of using features designed by hand. The model transforms the input x into a new representation $\varphi(x)$ in which the task becomes easier, for example in which the classes become linearly separable.

Raw pixels are a poor representation for four reasons. (1) They are high dimensional (784 values) and highly redundant, since neighbouring pixels are strongly correlated. (2) They are not invariant: shifting a lesion by a few pixels, or changing brightness or contrast between machines, changes many pixel values although the diagnosis is the same. (3) Distances in pixel space do not reflect meaning, so two scans of the same condition can be far apart while a normal and an abnormal scan can be close. (4) The decision depends on a few high-level patterns that are not visible in any single pixel, so the classes are not linearly separable in pixel space.

b) Show that no single straight line in the (x_1, x_2) space can separate class A from class B.

Class A = {(0,0), (1,1)} and class B = {(0,1), (1,0)}. Suppose a line $w_1x_1 + w_2x_2 + b = 0$ exists with class A on the negative side and class B on the positive side. Then:

- (0,0) ∈ A: $b < 0$
- (1,1) ∈ A: $w_1 + w_2 + b < 0$
- (0,1) ∈ B: $w_2 + b > 0$
- (1,0) ∈ B: $w_1 + b > 0$

Adding the two class A inequalities gives $w_1 + w_2 + 2b < 0$. Adding the two class B inequalities gives $w_1 + w_2 + 2b > 0$. These two results contradict each other, so no such w_1, w_2, b exists. (The same

contradiction arises if the sides are swapped.) Geometrically, the two class A points lie on one diagonal of the unit square and the two class B points on the other, so the segments joining each pair cross. This is the XOR problem, and it is not linearly separable.

c) Using the new representation $\varphi(x) = (x_1, x_2, x_1x_2)$, compute φ for the four points and show that $f = x_1 + x_2 - 2x_1x_2$ separates the two classes linearly in the new space.

Point (x_1, x_2)	Class	$\varphi(x) = (x_1, x_2, x_1x_2)$	$f = x_1 + x_2 - 2x_1x_2$	Side of f = 0.5
(0, 0)	A	(0, 0, 0)	0 + 0 − 0 = 0	Below
(1, 1)	A	(1, 1, 1)	1 + 1 − 2 = 0	Below
(0, 1)	B	(0, 1, 0)	0 + 1 − 0 = 1	Above
(1, 0)	B	(1, 0, 0)	1 + 0 − 0 = 1	Above

In the new space, $f = 1\cdot\varphi_1 + 1\cdot\varphi_2 - 2\cdot\varphi_3$ is a linear function of φ (weights (1, 1, −2), bias 0). It equals 0 for both class A points and 1 for both class B points. The plane $\varphi_1 + \varphi_2 - 2\varphi_3 = 0.5$ therefore separates the classes perfectly, even though no line could do so in the original space. This shows that a good representation (here, the extra feature x_1x_2) turns a non-separable problem into a linearly separable one, which is exactly what a hidden layer of a deep network learns to do.

d) Describe the hierarchy of features learned by successive layers of a deep network on chest X-ray images.

- **Layer 1 (low level):** edges, intensity gradients and small blobs, such as the boundary between lung and rib or bright and dark patches.
- **Layer 2 (mid level):** textures and simple shapes formed by combining edges, such as corners, curves, vessel-like lines and fine lung texture.
- **Layer 3 (part level):** anatomical structures and local abnormalities, such as the lung borders, the heart outline, round

nodule-like shapes and regions of increased opacity.

- **Top layers (high level):** whole-image patterns such as consolidation, pleural effusion or a mass, which feed the final normal/abnormal decision.

Each layer builds on the one below it, so complex concepts are composed from simple reusable parts. This is why the final decision, which depends on only a few high-level patterns, can be made reliably from 784 pixel values.

TASK 3 – Width and Depth of Neural Networks (4 Marks)

a) Calculate the number of trainable parameters in Network W and in Network D.

For a fully connected layer with n_in inputs and n_out outputs, the number of parameters is n_in × n_out + n_out (weights plus biases).

Network W (784 – 1000 – 10):

Layer	Calculation	Parameters
Input → Hidden (1000)	784 × 1000 + 1000	785,000
Hidden → Output (10)	1000 × 10 + 10	10,010
Total		795,010

Network D (784 – 128 – 128 – 128 – 10):

Layer	Calculation	Parameters
Input → Hidden 1 (128)	784 × 128 + 128	100,480
Hidden 1 → Hidden 2 (128)	128 × 128 + 128	16,512
Hidden 2 → Hidden 3 (128)	128 × 128 + 128	16,512
Hidden 3 → Output (10)	128 × 10 + 10	1,290
Total		134,794

Network W has **795,010** trainable parameters and Network D has

134,794, so Network W has about 5.9 times more parameters than Network D.

b) Explain the difference between the width and the depth of a network, and state what the universal approximation theorem says about a wide single-hidden-layer network.

The **width** of a network is the number of neurons in a layer (Network W has one layer 1,000 neurons wide). The **depth** is the number of layers through which the input is transformed (Network D has three hidden layers, so it is deeper). Width increases the number of features computed in parallel at one level, whereas depth increases the number of successive transformations, which allows features to be composed.

The universal approximation theorem states that a feedforward network with a single hidden layer containing a finite but sufficient number of neurons and a non-linear (non-polynomial) activation function can approximate any continuous function on a compact domain to any desired accuracy. The theorem guarantees that a wide shallow network can represent the function; it does not say how many neurons are required (this may be exponentially large), nor that gradient descent will find the weights, nor that the network will generalise from limited data.

c) Explain why a deep network can represent some functions with far fewer neurons than a shallow network, and name one training difficulty that increases with depth.

Depth allows reuse of intermediate features through composition. Each layer builds on the output of the previous one, so the number of distinct linear regions or patterns that the network can represent grows exponentially with depth but only polynomially with width. A classic example is the parity of n bits: a deep network can compute it with about O(n) neurons by combining bits in a chain or tree, whereas a single-hidden-layer network needs on the order of 2^{n-1} neurons. For images, a deep network learns edges once and reuses them in textures, parts and objects, while a shallow network must learn each complete pattern separately.

Training difficulty: the vanishing (or exploding) gradient problem. During backpropagation the gradient is multiplied by many layer

derivatives, so it can shrink toward zero in the early layers, which then learn very slowly. Optimisation also becomes harder and more sensitive to initialisation and learning rate as depth grows.

d) Recommend one of the two networks for 1,000 labelled scans and justify the choice using parameter count and generalisation.

Recommendation: Network D (deep and narrow). It has only 134,794 parameters against 795,010 for Network W, which means about 135 parameters per labelled scan against about 795. Both networks have more parameters than labelled samples, but Network W is much more over-parameterised, so it is far more likely to memorise the 1,000 scans and generalise poorly. Network D has lower capacity for memorisation, needs less memory, and is also more efficient in representing hierarchical patterns because of its depth. Its three layers learn edge → texture → pattern features that suit X-ray images, which improves generalisation. Its depth must be handled with ReLU-type activations, careful initialisation and regularisation (dropout, weight decay, early stopping), and ideally with unsupervised pre-training on the unlabelled scans.

TASK 4 – Activation Functions: ReLU, LReLU and ERELU (5 Marks)

a) Compute the output of ReLU, LReLU and ERELU for each of the four inputs, correct to four decimal places.

For ERELU with $\alpha = 1$ and $z \leq 0$: $f(z) = e^z - 1$. We have $e^{-3} = 0.049787$ and $e^{-1} = 0.367879$.

z	ReLU	LReLU (0.01z for z ≤ 0)	ERELU ($e^z - 1$ for z ≤ 0)
−3	0.0000	−0.0300	$e^{-3} - 1 = -0.9502$
−1	0.0000	−0.0100	$e^{-1} - 1 = -0.6321$
0	0.0000	0.0000	$e^{0} - 1 = 0.0000$
2	2.0000	2.0000	2.0000

b) Compute the derivative of each activation function at z = −3, −1 and 2.

ReLU: f′(z) = 0 for z < 0 and 1 for z > 0. LReLU: f′(z) = 0.01 for z ≤ 0 and 1 for z > 0. ERELU: f′(z) = α e^z = e^z for z ≤ 0 and 1 for z > 0.

z	ReLU′(z)	LReLU′(z)	ERELU′(z)
−3	0.0000	0.0100	e^{-3} = 0.0498
−1	0.0000	0.0100	e^{-1} = 0.3679
2	1.0000	1.0000	1.0000

c) Compare the three functions in terms of zero-centred output, smoothness, computational cost and behaviour for negative inputs.

Property	ReLU	LReLU	ERELU
Zero-centred output	No. Outputs are always ≥ 0, so the mean activation is positive.	Slightly better. Small negative outputs pull the mean closer to zero.	Best. Negative outputs (down to −1) push the mean activation closer to zero.
Smoothness	Not differentiable at z = 0 (sharp kink).	Not differentiable at z = 0, although the gradient never vanishes.	Smooth. With α = 1 the slopes on both sides of 0 are equal (1), so the first derivative is continuous.
Computational cost	Lowest. A single comparison.	Low. A comparison and one multiplication.	Highest. An exponential must be computed for negative inputs.
Negative inputs	Output is exactly 0 and the gradient is 0 (neuron is inactive).	Small linear output 0.01z and small constant gradient 0.01; unbounded below.	Output saturates smoothly towards −α = −1, with gradient e^z that decays but is non-zero.

d) State which function is most likely to produce neurons that stop learning and explain why.

ReLU is most likely to produce neurons that stop learning (the dying-ReLU problem). For $z \leq 0$ both its output and its gradient are exactly zero. If a neuron's pre-activation becomes negative for all inputs, for example after a large weight update, no gradient flows through it, its weights are never updated again, and it stays inactive permanently. LReLU keeps a small gradient (0.01) and ERELU keeps a gradient of e^z, so both can recover.

TASK 5 – Dying Neurons and Choice of Activation (3 Marks)

a) Calculate the largest possible pre-activation z and show that the neuron outputs zero for every input.

The pre-activation is $z = w_1x_1 + w_2x_2 + b = x_1 + x_2 - 3$, with $x_1, x_2 \in [0, 1]$. Since both weights are positive, z is largest when both inputs take their maximum value:

$$z_{max} = 1(1) + 1(1) - 3 = -1$$

Because $z \leq -1 < 0$ for every possible input, ReLU gives $f(z) = \max(0, z) = 0$ for all inputs. The neuron is permanently inactive (dead).

b) Explain, using the gradient, why such a neuron cannot recover during further training.

By the chain rule, the gradient of the loss L with respect to each weight is

$$\partial L/\partial w_i = (\partial L/\partial a) \cdot f'(z) \cdot x_i \qquad \text{and} \qquad \partial L/\partial b = (\partial L/\partial a) \cdot f'(z)$$

For $z < 0$, the ReLU derivative is $f'(z) = 0$, so every weight and bias gradient is exactly zero, whatever the upstream gradient $\partial L/\partial a$. The updates $w \leftarrow w - \eta \cdot 0$ leave the weights at (1, 1) and $b = -3$, so z stays at most -1 for every input and the neuron remains dead. Since the gradient is zero, there is no signal that could push the neuron back to a positive pre-activation, so it cannot recover on its own.

c) For z = −1, compare the gradient passed back by ReLU, LReLU (slope 0.01) and ERELU (α = 1), and suggest two other measures to avoid the problem.

Activation	Derivative at z = −1	Gradient passed back
ReLU	0	0.0000 (no learning)
LReLU (slope 0.01)	0.01	0.0100 × upstream gradient
ERELU (α = 1)	$\alpha\, e^{-1} = 0.3679$	0.3679 × upstream gradient

ReLU passes no gradient, LReLU passes a small gradient, and ERELU passes the largest gradient (about 37 times that of LReLU), so both LReLU and ERELU allow the neuron to recover. Two other measures to avoid dying neurons are:

- **Use a smaller learning rate** (with a learning-rate schedule or warm-up, or an adaptive optimiser such as Adam with gradient clipping), so that a single update does not push the bias and weights to extreme values.
- **Use proper initialisation and normalisation:** He initialisation keeps pre-activations in a sensible range at the start, and batch normalisation keeps the pre-activations centred so that they stay positive for a reasonable fraction of inputs. A small positive bias initialisation can also be used.

TASK 6 – Unsupervised Training of Neural Networks (3 Marks)

a) Define unsupervised training of a neural network and state what the training objective looks like when no labels are available.

Unsupervised training is the training of a neural network using only the inputs x, without target labels, so that the network discovers structure in the data, such as compact features, clusters or the data distribution. Since there is no label y to compare with, the objective is defined on the input itself. For an auto encoder, the network reconstructs its own input and the objective is the reconstruction error:

$$L = (1/N)\ \Sigma_n\ \|x_n - g(f(x_n))\|^2$$

where f is the encoder and g is the decoder. For an RBM, the objective is to maximise the log-likelihood of the data, $\Sigma_n \log P(v_n)$, or

equivalently to minimise the negative log-likelihood, which is the same as lowering the energy of the training data. In both cases the "target" is the data itself.

b) Explain greedy layer-wise unsupervised pre-training followed by supervised fine-tuning, listing the steps in order.

- **Step 1:** Train the first layer (an RBM or a shallow auto encoder) on the unlabelled scans (the 784 inputs) to learn a first set of features h_1.
- **Step 2:** Freeze the first layer. Pass all the scans through it to obtain h_1, and use h_1 as the input to train a second layer (RBM/auto encoder) without labels, producing h_2.
- **Step 3:** Repeat the process for each additional layer, training one layer at a time (greedy) on the output of the one below it.
- **Step 4:** Stack the trained encoders to form a deep network, and initialise its weights with the learned values.
- **Step 5:** Add a randomly initialised output layer (normal/abnormal) on top of the final hidden layer.
- **Step 6:** Fine-tune the whole network with supervised back-propagation on the 1,000 labelled scans, using a small learning rate so that the pre-trained features are refined and not destroyed.

c) Explain two benefits of unsupervised pre-training when labelled data is scarce.

- **Better initialisation and optimisation:** the weights start in a region of parameter space that already captures the structure of chest X-rays, so supervised training starts from a good point, converges faster, and avoids poor local minima or vanishing gradients that a random start may encounter in a deep network.
- **Reduced overfitting and efficient use of data:** the 100,000 unlabelled scans supply far more information about the input distribution than 1,000 labelled scans. The pre-training acts as a data-driven regulariser, so fine-tuning only needs to adjust the features for the classification task, which improves generalisation to new patients.

TASK 7 – Restricted Boltzmann Machines: Energy and Inference (5 Marks)

a) Write the joint probability P(v, h) in terms of the energy and explain the role of the partition function Z.

$$P(v, h) = e^{-E(v, h)} / Z, \qquad \text{where} \qquad Z = \Sigma_v \Sigma_h e^{-E(v, h)}$$

States with low energy have high probability. The partition function Z is the sum of e^{-E} over all possible joint configurations of visible and hidden units. It is a normalising constant that guarantees that the probabilities sum to 1. For 3 visible and 2 hidden binary units there are $2^5 = 32$ terms, but for realistic RBMs (such as 784 visible units) the number of terms is exponentially large, so Z cannot be computed exactly. This intractability is why approximate learning methods such as contrastive divergence are used.

b) Compute E(v, h) for v = (1, 0, 1) and h = (1, 0).

The energy is $E(v, h) = -a^T v - b^T h - v^T W h$. Each term is computed separately:

- $a^T v = 0.1(1) + 0(0) + (-0.1)(1) = 0$, so $-a^T v = 0$.
- $b^T h = (-0.5)(1) + 0.2(0) = -0.5$, so $-b^T h = +0.5$.
- $v^T W$ = row 1 + row 3 = $(1.0 - 1.0, -0.5 + 0.5) = (0, 0)$, so $v^T W h = (0, 0)\cdot(1, 0) = 0$.

$$E(v, h) = 0 + 0.5 + 0 = \mathbf{0.5}$$

c) Compute $P(h_1 = 1 \mid v)$ and $P(h_2 = 1 \mid v)$ using the sigmoid function.

For an RBM, $P(h_j = 1 \mid v) = \sigma(b_j + \Sigma_i v_i W_{ij})$, where $\sigma(x) = 1/(1 + e^{-x})$. Using $v^T W = (0, 0)$:

- Hidden unit 1: net input = $-0.5 + 0 = -0.5$, so $P(h_1 = 1 \mid v) = \sigma(-0.5) = 1/(1 + e^{0.5}) = 1/2.6487 = \mathbf{0.3775}$.
- Hidden unit 2: net input = $0.2 + 0 = 0.2$, so $P(h_2 = 1 \mid v) = \sigma(0.2) = 1/(1 + e^{-0.2}) = 1/1.8187 = \mathbf{0.5498}$.

d) Explain why the bipartite structure of an RBM makes the hidden units conditionally independent given v and enables efficient block Gibbs sampling.

In an RBM the connections exist only between visible and hidden units; there are no visible–visible or hidden–hidden connections (the graph is bipartite). When v is fixed, the energy is a sum of terms

each involving a single hidden unit, $E = -a^{T}v - \sum_j h_j(b_j + \sum_i v_i W_{ij})$, so e^{-E} factorises over j. Hence $P(h \mid v) = \prod_j P(h_j \mid v)$, and each hidden unit is a simple sigmoid of its own net input. By the same argument the visible units are conditionally independent given h.

Because of this, all hidden units can be sampled in parallel in one step given v, and then all visible units can be sampled in parallel given h. This alternating, block-wise procedure is block Gibbs sampling. It needs only a matrix multiplication and a sigmoid per block (no unit-by-unit sequential updating), which is fast and suited to GPUs, and makes learning with contrastive divergence practical.

TASK 8 – Restricted Boltzmann Machines: Contrastive Divergence (4 Marks)

a) Compute the reconstruction probabilities $P(v_i = 1 \mid h)$ for i = 1, 2, 3.

For h = (1, 0), $P(v_i = 1 \mid h) = \sigma(a_i + \sum_j W_{ij} h_j) = \sigma(a_i + W_{i1})$, since $h_2 = 0$.

Unit	Net input	$P(v_i = 1 \mid h)$
v_1	0.1 + 1.0 = 1.1	$\sigma(1.1) = 1/(1 + e^{-1.1}) = 0.7503$
v_2	0 + 0.5 = 0.5	$\sigma(0.5) = 1/(1 + e^{-0.5}) = 0.6225$
v_3	−0.1 + (−1.0) = −1.1	$\sigma(-1.1) = 1/(1 + e^{1.1}) = 0.2497$

b) Suppose the sampled reconstruction is v′ = (1, 1, 0). Compute $P(h' = 1 \mid v')$ for both hidden units.

With v′ = (1, 1, 0): net input to hidden unit j = $b_j + 1 \cdot W_{1j} + 1 \cdot W_{2j} + 0 \cdot W_{3j}$.

- Hidden unit 1: −0.5 + 1.0 + 0.5 = 1.0, so $P(h'_1 = 1 \mid v') = \sigma(1.0) =$ **0.7311**.
- Hidden unit 2: 0.2 + (−0.5) + 1.0 = 0.7, so $P(h'_2 = 1 \mid v') = \sigma(0.7) =$ **0.6682**.

c) Using $\Delta W_i = \varepsilon(v_i h - v'_i P(h' = 1 \mid v'))$, compute ΔW_{11} and

ΔW_{31}.

Here $\varepsilon = 0.1$, the data vector is $v = (1, 0, 1)$, the sampled hidden state is $h = (1, 0)$ (so $h_1 = 1$), and $P(h'_1 = 1 \mid v') = 0.7311$. The first index is the visible unit and the second is the hidden unit.

$$\Delta W_{11} = 0.1\,(v_1 h_1 - v'_1\, P(h'_1 = 1 \mid v')) = 0.1\,(1 \times 1 - 1 \times 0.7311) = 0.1 \times 0.2689 = \mathbf{0.0269}$$

$$\Delta W_{31} = 0.1\,(v_3 h_1 - v'_3\, P(h'_1 = 1 \mid v')) = 0.1\,(1 \times 1 - 0 \times 0.7311) = 0.1 \times 1 = \mathbf{0.1000}$$

W_{11} increases by 0.0269 (the reconstruction already matches the data on v_1, so the correction is small), while W_{31} increases by 0.1 because the reconstruction failed to switch on v_3, so the data association between v_3 and h_1 has to be strengthened.

d) Explain why the exact gradient of the log-likelihood is intractable and how CD-1 approximates it.

The exact gradient of the log-likelihood of a data vector with respect to a weight is

$$\partial \log P(v) \,/\, \partial W_{ij} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$$

The first term (data expectation) is easy to compute because $P(h \mid v)$ factorises. The second term (model expectation) is an average over the model's joint distribution $P(v, h) = e^{-E}/Z$, which requires the partition function Z and a sum over all 2^{m+n} configurations of the m visible and n hidden units. This is exponentially expensive, so the exact gradient is intractable.

Contrastive divergence replaces the model expectation by a sample obtained after only a short Gibbs chain started at the data. In CD-1: (1) clamp v to a training vector and sample $h \sim P(h \mid v)$; (2) reconstruct $v' \sim P(v \mid h)$; (3) compute $P(h' \mid v')$; (4) update $\Delta W_{ij} = \varepsilon(\langle v_i h_j \rangle_{\text{data}} - \langle v'_i h_j' \rangle_{\text{recon}})$. This is a biased but low-variance and cheap approximation, and it works well in practice because the reconstruction is a sample from near the data where the model's probability needs to be corrected.

TASK 9 – Auto Encoders: Architecture and Reconstruction (4 Marks)

a) For the 784–128–784 auto encoder, calculate the number of

trainable parameters (including biases) and the compression ratio.

Layer	Calculation	Parameters
Encoder: 784 → 128	784 × 128 + 128	100,480
Decoder: 128 → 784	128 × 784 + 784	101,136
Total		201,616

The compression ratio is the input size divided by the code size: 784 / 128 = **6.125 : 1**. The code is only about 16.3% of the size of the input.

b) For the 2–1–2 auto encoder, compute the code h and the reconstruction $\hat{x}$, and calculate the loss $L = \frac{1}{2} \Sigma (x_i - \hat{x}_i)^2$.

Input x = (2, 4). The hidden pre-activation is z = 0.5(2) + 0.25(4) + 0 = 1 + 1 = 2. With ReLU, the code is

$$h = \max(0, 2) = \mathbf{2}$$

The linear output layer gives $\hat{x}$ = (0.8h, 1.6h) = (0.8 × 2, 1.6 × 2) = **(1.6, 3.2)**. The loss is

$$L = \frac{1}{2} [(2 - 1.6)^2 + (4 - 3.2)^2] = \frac{1}{2} [0.16 + 0.64] = \frac{1}{2} (0.80) = \mathbf{0.40}$$

c) Explain what can happen if the hidden layer is larger than the input without any constraint, and how an undercomplete bottleneck prevents it.

If the hidden layer is larger than the input (overcomplete) and there is no constraint, the auto encoder can learn the trivial identity function, copying the input to the output through the hidden layer. It then gets a near-zero reconstruction error without learning any useful structure, and it will also copy any noise, so the code is not a meaningful representation.

An undercomplete bottleneck (hidden size smaller than the input, as in 784 → 128) forces the network to pass the information through a small code. It cannot copy all 784 pixel values, so it has to keep only the most important, systematic variations in the scans and discard noise and redundancy. The code therefore becomes a compact, useful feature representation.

d) State the relationship between a linear auto encoder trained with squared error and Principal Component Analysis.

A linear auto encoder (no non-linear activations) with a k-unit hidden layer trained to minimise the squared reconstruction error learns to project the data onto the same k-dimensional subspace that is spanned by the first k principal components of PCA, which is the subspace that minimises the squared reconstruction error. The learned weight vectors need not be orthogonal or ordered by variance, but they span the same subspace. With non-linear activations and several layers, the auto encoder generalises PCA to a non-linear manifold and can capture more complex structure.

TASK 10 – Auto Encoder Variants (3 Marks)

a) Write the training objective of a denoising auto encoder and explain how corrupting the input helps the network learn robust features.

A denoising auto encoder (DAE) corrupts the input x into $\tilde{x}$ by drawing from a corruption distribution $q(\tilde{x} \mid x)$, for example adding Gaussian noise $\tilde{x} = x + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2 I)$ or randomly setting some pixels to zero (masking). It then reconstructs the original clean input from the corrupted one:

$$L_{DAE} = E_{\tilde{x} \sim q(\tilde{x} \mid x)} \| x - g(f(\tilde{x})) \|^2$$

The target is the clean x, not $\tilde{x}$. The network cannot simply copy its input, because the input contains noise, so it must learn the structure of the data (the correlations between pixels and the shape of normal anatomy) in order to remove the noise and fill in the missing parts. The features learned in this way are stable to small perturbations, which makes them robust.

b) For the sparse auto encoder with an L1 penalty, compute the total loss = reconstruction loss + $\lambda \Sigma |h|$.

Given: reconstruction loss = 0.40, h = (0.8, 0, 0.1, 0) and $\lambda = 0.5$. Then $\Sigma|h| = 0.8 + 0 + 0.1 + 0 = 0.9$, so the penalty is $\lambda \Sigma|h| = 0.5 \times 0.9 = 0.45$.

$$\text{Total loss} = 0.40 + 0.45 = \mathbf{0.85}$$

c) Choose the better variant for noisy scans from older

machines and justify the choice.

The **denoising auto encoder** is the better choice. Its objective is explicitly to recover the clean signal from a corrupted input, so by training it with noise that resembles the noise of the older machines it learns features that ignore that noise and capture true anatomy and lesions. A sparse auto encoder only encourages few active hidden units; it does not, by itself, make the model robust to noise, and it can still encode noise if the noise is the dominant variation. In practice the two can be combined (a sparse denoising auto encoder), but for noisy input, denoising is the primary requirement.

TASK 11 – Deep Learning Applications (4 Marks)

a) Match each of the three uses to a suitable deep learning architecture and justify each choice.

Use	Architecture	Justification
Classifying scans (normal/abnormal)	Convolutional Neural Network (CNN), possibly pre-trained and fine-tuned	Convolution and weight sharing exploit the spatial structure of images, detect lesions regardless of position, and need far fewer parameters than a fully connected network.
Detecting unusual scans (trained only on normal scans)	Convolutional auto encoder	It learns to reconstruct normal scans; abnormal scans are reconstructed poorly, so the reconstruction error acts as an anomaly score without needing abnormal labels.
Extracting findings from radiologist text reports	Transformer-based language model (BERT-style) or LSTM/RNN	Text is sequential and context-dependent; these models capture long-range context and are suited to named-entity recognition and information extraction.

b) Calculate the threshold and state which of the four scans are flagged.

$$\text{Threshold} = \mu + 3\sigma = 0.03 + 3(0.01) = \mathbf{0.06}$$

Scan error	Compared	Distance from	Decision

	with 0.06	mean (σ units)	
0.02	Below	−1σ	Not flagged (normal)
0.05	Below	+2σ	Not flagged (normal)
0.07	Above	+4σ	**Flagged**
0.31	Above	+28σ	**Flagged**

The scans with errors **0.07 and 0.31** are flagged as unusual. The scan with 0.31 is extremely far from the normal range, whereas 0.07 is just above the threshold and should be reviewed by a radiologist.

c) Explain why only normal scans are used to train the detector.

Abnormalities are rare and extremely varied (many diseases, sizes and locations), so it is impossible to collect labelled examples of all of them, whereas normal scans are plentiful. If the auto encoder is trained only on normal scans, it learns the manifold of normal anatomy and can reconstruct normal scans well (low error). A scan with an abnormality does not lie on this manifold, so it is reconstructed poorly and gives a high error that can be thresholded. If abnormal scans were also included in training, the network would learn to reconstruct them too, and the gap in error between normal and abnormal scans would shrink, so the detector would miss new, unseen abnormalities.

d) Discuss two practical concerns, such as privacy, interpretability or bias, in deploying deep learning in healthcare.

- **Privacy:** scans and reports are sensitive patient data governed by laws and regulations (for example, HIPAA, GDPR and India's data protection law). Models can also leak information about their training data. The hospital must de-identify the images and text, restrict access, and consider federated learning or differential privacy when training across several hospitals.
- **Interpretability and bias:** a black-box model that flags a scan without explanation is hard for radiologists to trust or to verify, and errors can harm patients. Visual explanations (Grad-CAM, saliency maps), reconstruction-error maps and a

human-in-the-loop review are needed. In addition, a model trained on data from limited hospitals, scanners or patient groups may perform worse for other groups, so performance should be audited across sub-populations and devices.

TASK 12 – Design Challenge: Selecting and Adapting a Deep Learning Architecture (8 Marks)

Changed requirement: the original network was one hidden layer of 1,000 sigmoid neurons trained on 1,000 labelled 28 × 28 scans. The new data are 200,000 unlabelled and 2,000 labelled 256 × 256 scans from five hospitals, with noisy images, rare abnormalities and limited-memory devices. Decision: **redesign** the architecture.

A. Impact Analysis (2 Marks)

a) Explain how 256 × 256 inputs, noisy images and scarce labels affect the parameter count, training and generalisation of the original network.

Parameter count: a 256 × 256 image has 65,536 pixels. The first layer of the original network would have 65,536 × 1,000 + 1,000 = 65,537,000 parameters, and with a 2-class output layer (1,000 × 2 + 2 = 2,002) the total is about **65.54 million**. The original 28 × 28 network had 785,000 + 2,002 = 787,002 parameters, so the model grows about 83 times. In 32-bit floating point this needs about 262 MB of memory.

Training: the number of weights to be estimated increases enormously, so each epoch is much slower and needs much more memory. Sigmoid units in a layer of this size are easily saturated, which slows learning.

Generalisation: with 2,000 labelled scans there are about 32,800 parameters per labelled scan, so the network can easily memorise the labels (overfit). A fully connected layer treats every pixel independently, so noise in individual pixels is fitted as if it were a signal, and differences between the five hospitals' scanners (domain shift) can degrade the results on new sites. Rare abnormalities are also under-represented, so the classifier may be biased toward predicting "normal".

b) Discuss why representation learning, depth and activation

behaviour become important.

- **Representation learning:** 200,000 unlabelled scans are available, which is a rich source for learning features of chest anatomy without labels. Learned features that are compact and noise-robust replace the 65,536 raw pixels as input, which reduces dimensionality and the amount of labelled data needed.
- **Depth:** a hierarchy of layers (edges → textures → lesion patterns) represents the structure of large images with far fewer neurons than one very wide layer (Task 3c), and convolutional depth allows weight sharing.
- **Activation behaviour:** deeper networks need activations that keep gradients alive. Sigmoid gradients are at most 0.25, so they vanish through many layers, while ReLU, LReLU or ERELU keep a gradient of 1 for positive inputs (and a small non-zero gradient for negative inputs in LReLU/ERELU), which makes training deep networks and avoiding dead neurons feasible.

B. Evaluation of the Original Approach (2 Marks)

a) Evaluate whether a single-hidden-layer sigmoid network is sufficient for the new requirement.

It is **not sufficient**. Although the universal approximation theorem says that a wide single-hidden-layer network could in principle represent the required function, this says nothing about whether it can be learned from 2,000 labelled noisy scans, and the width needed would be impractically large. The network ignores the 2-D structure of the image, uses none of the 200,000 unlabelled scans, has no mechanism for handling noise or rare classes, and is too large for devices with limited memory.

b) Explain its limitations in terms of capacity, vanishing gradients, label scarcity and memory.

- **Capacity:** one hidden layer cannot form a hierarchy of features. To model complex anatomy and subtle lesions it would need an enormous number of neurons, and it has no translation invariance, so a lesion at a different location must be learned separately.
- **Vanishing gradients:** the sigmoid derivative is $\sigma'(z) = \sigma(z)(1 -$

$\sigma(z)) \leq 0.25$ and approaches 0 when the unit saturates. Even in the single layer, saturated units learn very slowly, and if depth is added the gradient shrinks by up to a factor 0.25 per layer; for five layers this is at most $0.25^5 \approx 0.001$, so early layers barely learn.

- **Label scarcity:** about 65.5 million parameters against 2,000 labels make overfitting almost certain, and the network cannot exploit the 200,000 unlabelled scans.
- **Memory:** about 262 MB of weights (float32) is too large for limited-memory devices, and inference on 65,536 inputs per image is costly.

C. Adaptation or Redesign (2 Marks)

a) Propose a redesigned architecture using unsupervised pre-training (an auto encoder or RBM stack), a suitable depth and width, and a suitable activation function such as ReLU, LReLU or ERELU.

The proposal is a **convolutional denoising (sparse) auto encoder** pre-trained on all 200,000 unlabelled scans, whose encoder is then reused with a small classification head. Convolutions with stride 2 reduce the image from 256 × 256 to 8 × 8 and keep the parameter count small through weight sharing. Hidden layers use **LReLU (slope 0.01)** (or ERELU if the extra exponential cost is acceptable) to avoid dying neurons while keeping low cost, with He initialisation and batch normalisation. The encoder has five convolutional layers with 3 × 3 kernels, then a 128-unit code.

Layer	Output size	Calculation	Parameters
Conv1 (1→16, stride 2)	128 × 128 × 16	3·3·1·16 + 16	160
Conv2 (16→32, stride 2)	64 × 64 × 32	3·3·16·32 + 32	4,640
Conv3 (32→64, stride 2)	32 × 32 × 64	3·3·32·64 + 64	18,496
Conv4 (64→64, stride 2)	16 × 16 × 64	3·3·64·64 + 64	36,928
Conv5 (64→64, stride 2)	8 × 8 × 64	3·3·64·64 + 64	36,928

Layer	Output size	Calculation	Parameters
Code (FC 4,096→128)	128	4,096·128 + 128	524,416
Encoder total			621,568
Classifier head (128→64→2)	2	(128·64 + 64) + (64·2 + 2)	8,386
Deployed network			629,954

The deployed classifier (encoder + head) has about **0.63 million** parameters, roughly **104 times fewer** than the original 65.54 million, and needs about 2.5 MB in float32 (about 0.63 MB after 8-bit quantisation). The decoder (a mirror of the encoder using transposed convolutions) is used only during pre-training and for the anomaly detector, so it does not add to the deployed classifier. The code compresses 65,536 pixels into 128 values (a ratio of 512 : 1).

b) Explain how a denoising or sparse auto encoder, supervised fine-tuning and the size of the bottleneck code can be incorporated.

- **Denoising:** during pre-training, each scan is corrupted (for example Gaussian noise plus random pixel masking that mimics older machines) and the auto encoder is trained to reconstruct the clean scan, $L = \|x - g(f(\tilde{x}))\|^2$. The encoder then learns noise-robust features. Scans from all five hospitals are used so that the features cover several scanner types.
- **Sparse penalty:** an L1 term is added to the loss on the code, $L_{total} = L_{rec} + \lambda \Sigma|h|$ (as in Task 10), to encourage a few active code units and more selective features. λ is tuned on the validation set.
- **Bottleneck size:** the code size is a trade-off. Too small (for example 16) loses fine lesion detail; too large (for example 1,024) lets the network copy noise (Task 9c). Sizes of 32, 64, 128 and 256 are compared on validation reconstruction error and classification AUROC, and 128 is the starting choice.
- **Supervised fine-tuning:** after pre-training, the decoder is discarded and the classification head is attached. First, only the

head is trained on the 2,000 labelled scans with the encoder frozen; then the whole network is fine-tuned with a small learning rate, dropout, weight decay, data augmentation and early stopping. Class weighting or oversampling handles the rare abnormal class.

- **Rare abnormalities:** the same auto encoder, trained only on normal scans, can run alongside the classifier and flag scans whose reconstruction error exceeds $\mu + 3\sigma$ (Task 11) for review.

D. Evidence-Based Defence (2 Marks)

a) Propose experiments and evaluation measures to compare the redesigned approach with the original one.

- **Baselines:** (i) original sigmoid network on down-sampled inputs, (ii) the same CNN trained from scratch without pre-training, and (iii) the proposed pre-trained model. All use the same 2,000 labelled scans.
- **Data splits:** hold out a test set, and use leave-one-hospital-out evaluation (train on four hospitals and test on the fifth) to measure generalisation across sites.
- **Metrics:** because abnormal cases are rare, use sensitivity (recall), specificity, precision, F1-score, AUROC and AUPRC, not plain accuracy; add the confusion matrix and calibration. For the detector, report detection rate and false-alarm rate at the $\mu + 3\sigma$ threshold.
- **Label efficiency:** train with 250, 500, 1,000 and 2,000 labels and plot performance against the number of labels to show the benefit of pre-training.
- **Robustness to noise:** test with added Gaussian and masking noise at increasing levels, and report how much performance degrades.
- **Ablations:** ReLU vs LReLU vs ERELU vs sigmoid; with and without denoising; with and without the sparsity penalty; bottleneck sizes 32 to 256; with and without pre-training.
- **Deployment measures:** parameter count, model size in MB, peak memory and inference time per scan on the target device, before and after quantisation.
- **Reliability:** repeat each experiment with several random seeds

and report mean ± standard deviation, with confidence intervals or a paired significance test.

b) Justify the final design using parameter count, generalisation, robustness to noise and deployment constraints.

Criterion	Original (1 sigmoid layer, 256 × 256 input)	Redesigned (pre-trained conv. denoising auto encoder + LReLU)
Parameter count	≈ 65.54 million	≈ 0.63 million (about 104× fewer)
Memory	≈ 262 MB (float32)	≈ 2.5 MB (float32); ≈ 0.63 MB (int8)
Use of data	Only 2,000 labelled scans	200,000 unlabelled scans for pre-training plus 2,000 labelled for fine-tuning
Generalisation	About 32,800 parameters per label, so severe overfitting is likely; ignores image structure	About 315 parameters per label and shared convolutional weights; pre-trained features and regularisation
Robustness to noise	No mechanism; noise is fitted as signal	Denoising training plus sparsity give noise-robust features; multi-hospital data reduce domain shift
Gradient flow	Saturating sigmoid; gradients ≤ 0.25	LReLU keeps a gradient of 1 (or 0.01), so there are no dead or vanishing units
Rare abnormalities	Biased toward the majority class	Class weighting plus an auto encoder anomaly detector trained on normal scans

In conclusion, the original network should be **redesigned**, not retained or merely modified. The proposed model is about 100 times smaller, fits on limited-memory devices, uses the large unlabelled collection to learn robust representations, is better protected against overfitting, and is more robust to noise and to differences between hospitals. The evaluation plan above would confirm or reject these

expectations with measurable evidence before deployment.